City Tour Planner — Midterm Design Document

Course: COMP.SE.110 Software Design, 2026–2027

Submission: Midterm Design Document

Team: Varun Ganesh Sondur, Pham Thanh, Georgios Malezoglou, Abdullah

Table of Contents

1. High-level description 2
2. Architecture overview 3
3. Boundaries and interfaces 6
4. Components and responsibilities 7
5. Data sources and design decisions 8
6. API contract (summary) 9
7. Implementation status 10
8. Self-evaluation 10
9. How to run the application 14

1. High-level description

City Tour Planner is a web application that generates a short city tour by combining data from several independent sources. It receives input from the user in terms of the city name, date, time of starting and tour duration. Based on this information, the tool fetches weather data, traffic data and places information to create a basic itinerary.

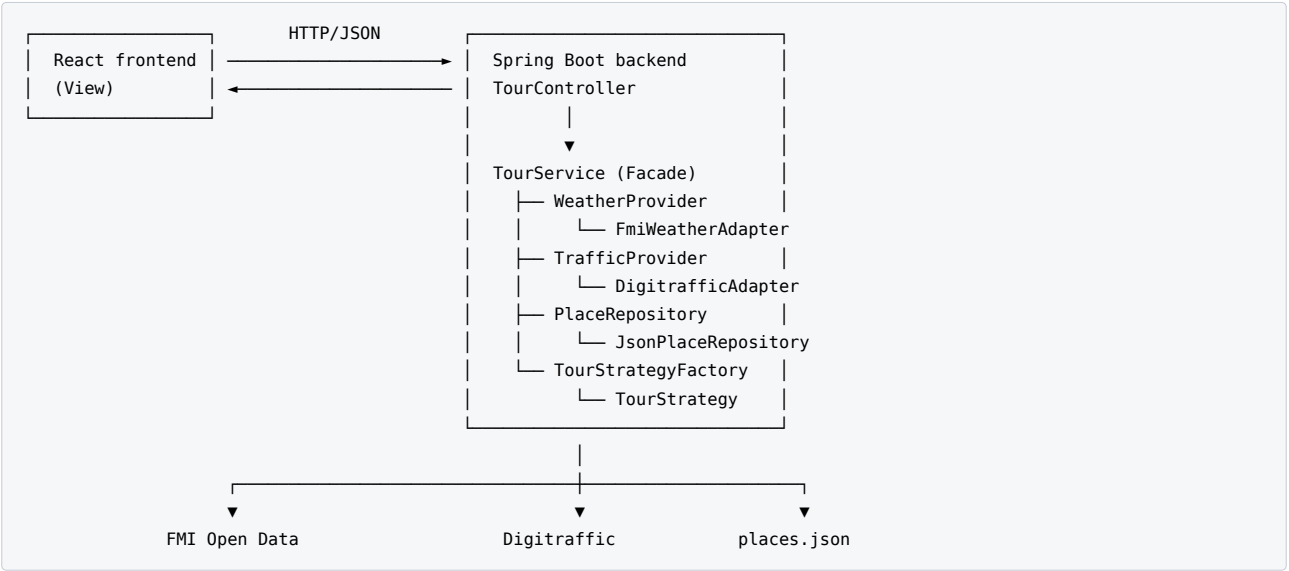
Goals

- Extract information from various sources independently.
- Combine the data in a meaningful way.
- Create an initial schedule.
- Visualise the result (map, chart, stop list, traffic).
- Make the design flexible to include more data sources in the future.

Technology stack

Layer	Technology	Why
Backend	Java 22, Spring Boot 3.5, Maven	Strong typing, mature HTTP ecosystem, easy REST + validation
Frontend	React 19, Vite, Tailwind CSS 4, Leaflet, Recharts	Fast UI development, good map and chart libraries, pure view layer
Weather	FMI Open Data (WFS)	Official Finnish weather data, free, no key required
Traffic	Digitraffic	Official road traffic announcements
Places	Local JSON (Visit Tampere data)	No suitable open API covered all needed places; TA approved the change

High-level structure



The frontend serves as the presentation layer, which is responsible for handling forms, display state and visualization of data locally, whereas tour planning policy, partial failure and data integration belong to the backend. The frontend to backend interaction depicted above is the **target architecture**. The React UI is currently using sample data during midterm.

2. Architecture overview

Design patterns used

Pattern	Where	Role
Facade	`TourService`	Single entry point for tour generation; orchestrates providers and handles partial failure
Adapter	`FmiWeatherAdapter`, `DigitrafficAdapter`	Turn external APIs into the domain interfaces `WeatherProvider` and `TrafficProvider`
Strategy	`TourStrategy` + `TourStrategyFactory`	Different stop-selection behaviours depending on weather/traffic factors
Strategy-selection factory	`TourStrategyFactory.choose()`	Selects `GeneralTourStrategy` or `IndoorTourStrategy` based on forecast and factors
Repository	`PlaceRepository` / `JsonPlaceRepository`	Abstract place storage; currently a local JSON file
Proxy (planned)	Caching layer in front of providers	Avoid repeated calls for the same city/date when the user changes language or planning mode

Diagrams

PlantUML sources are in `backend/docs/`:

- `high-level-design.puml` — overall component picture
- `interfaces-class-diagram.puml` — interfaces, models and relationships
- `generate-tour-sequence.puml` — sequence for page load and generate-tour

The following are the team's existing PlantUML diagrams from `backend/docs/`, included here for convenience to allow reviewing the design in one document. The `.puml` files are the actual source; use PlantUML to render them into a PDF. This sequence represents the **desired integrated flow** rather than the current implementation in the React frontend.

High-level component diagram

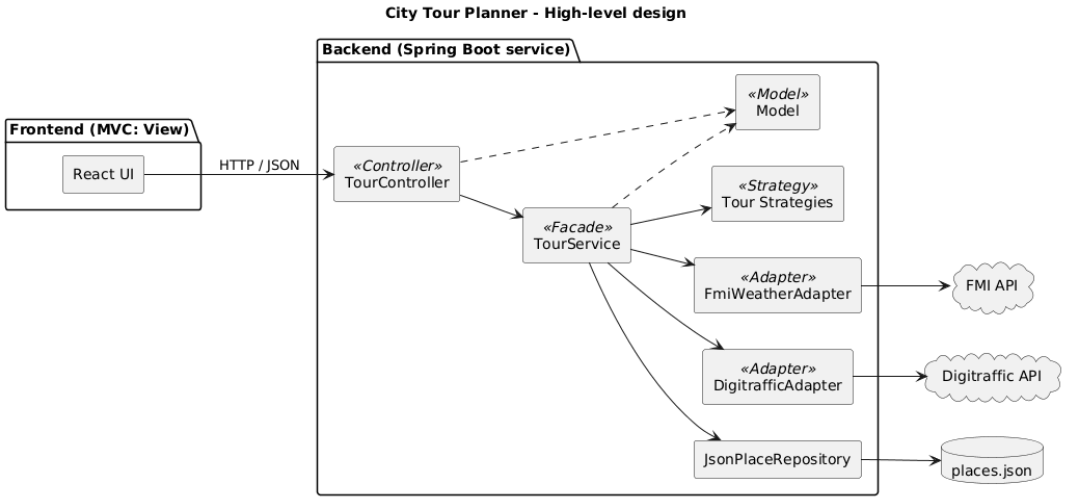


Figure 1. High-level component diagram.

Interfaces and class relationships

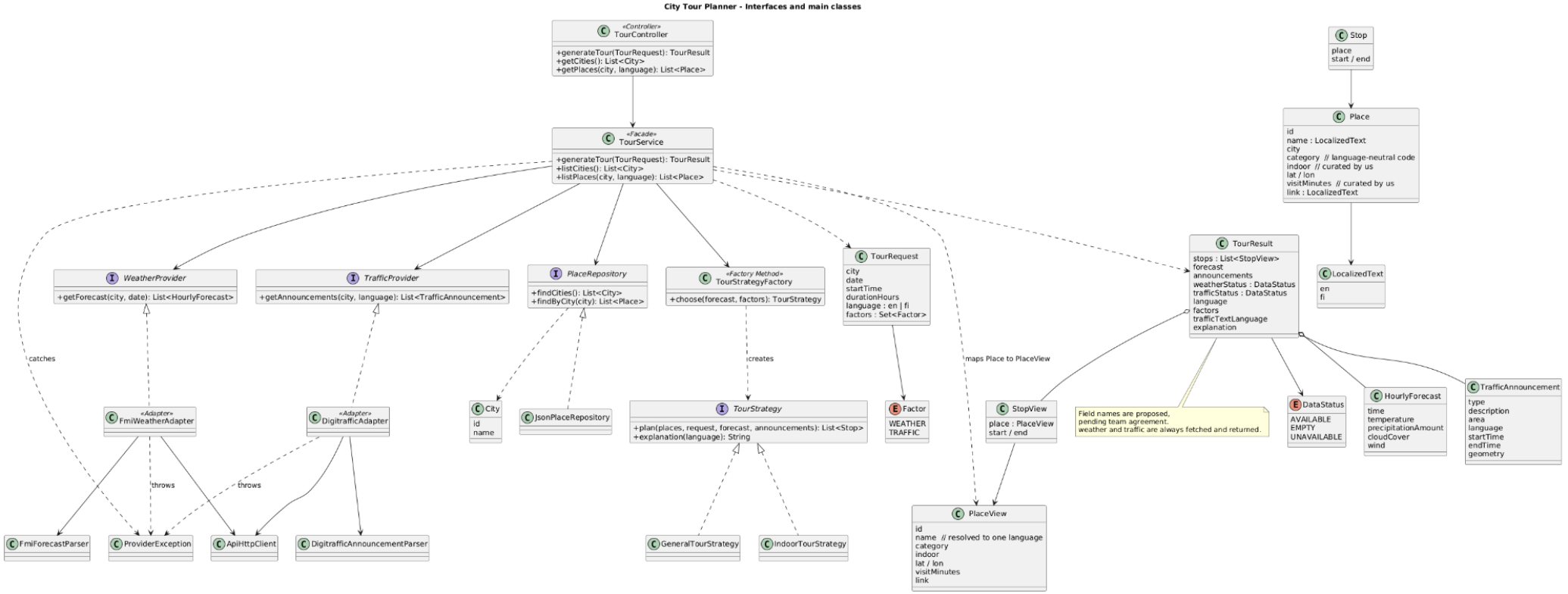


Figure 2. Interfaces and class relationships.

Planned request sequence

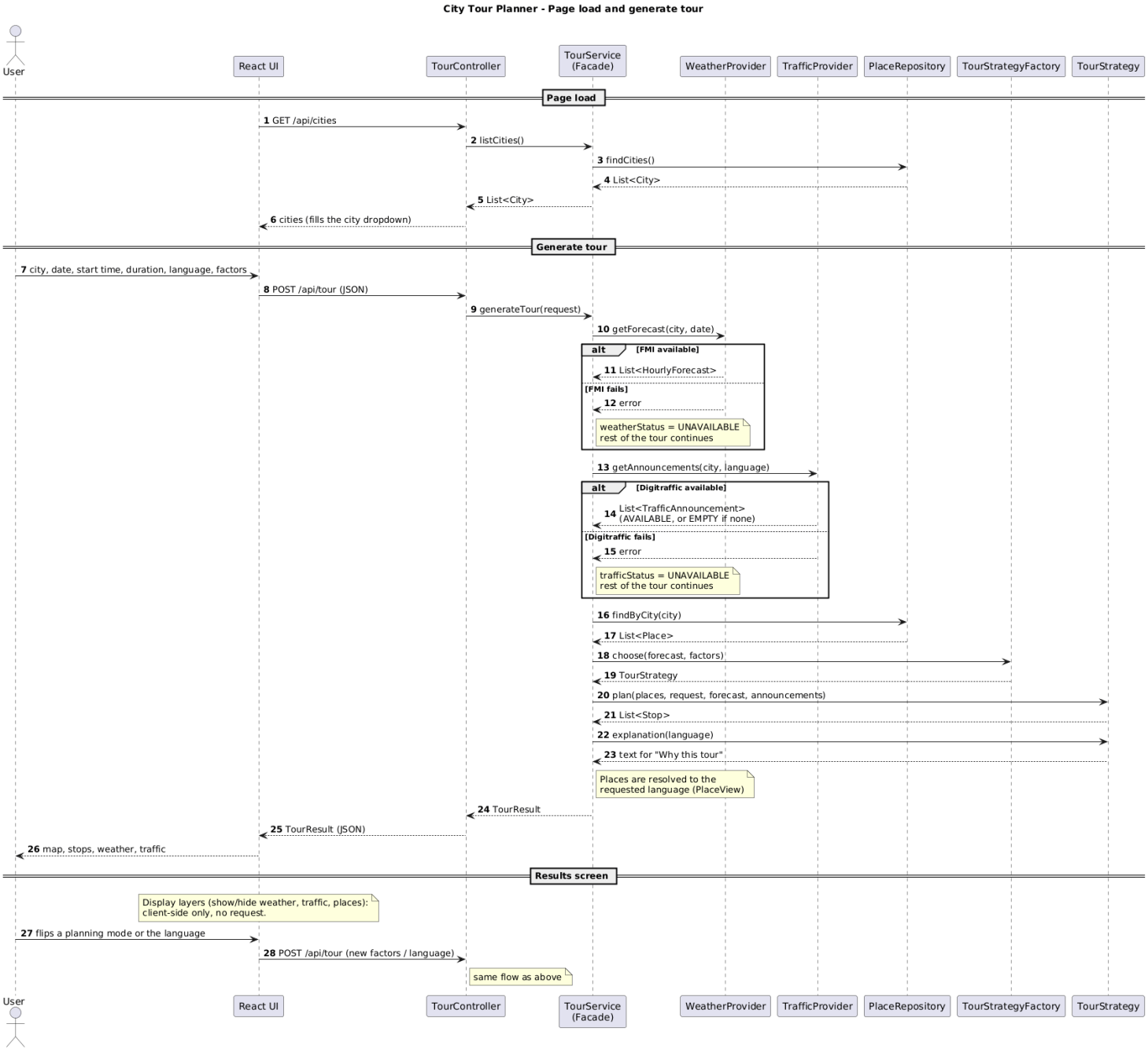


Figure 3. Planned request sequence.

3. Boundaries and interfaces

External boundaries

Boundary	Protocol	Notes
Frontend ↔ Backend	HTTP JSON on port 8080	CORS allows `http://localhost:5173`
Backend ↔ FMI	HTTPS WFS	XML multipoint coverage forecast
Backend ↔ Digitraffic	HTTPS REST	Traffic announcements
Backend ↔ Places	Classpath JSON file	No network call

Internal interfaces (backend)

```
WeatherProvider
  List\<HourlyForecast> getForecast(City city, LocalDate date) throws ProviderException
TrafficProvider
  List\<TrafficAnnouncement> getAnnouncements(City city, Language language) throws ProviderException
PlaceRepository
  List\<City> findCities()
  List\<Place> findByCity(String cityId)
TourStrategyFactory
  TourStrategy choose(List\<HourlyForecast> forecast, List\<Factor> factors)
TourStrategy
  List\<Stop> plan(List\<Place> places, TourRequest request,
                 List\<HourlyForecast> forecast,
                 List\<TrafficAnnouncement> announcements)
  String explanation(Language language)
```

- **Demand side:** `TourService` (Facade) depends only on the interfaces above.
- **Supply side:** adapters and the JSON repository implement those interfaces.
- HTTP details live in `ApiHttpClient` / `ApiRequestFactory`; adapters never build raw URLs themselves.

Information flow (generate tour)

1. UI sends `POST /api/tour` with city, date, start time, duration, language, factors.
2. `TourController` validates and calls `TourService.generateTour`.
3. `TourService` invokes weather and traffic providers (in case of failure it converts exceptions to status flags while further tour generation continues).
4. Places are loaded from the repository.
5. Factory selects a strategy that generates ordered list of stops.
6. Result is assembled as `TourResult` (stops + weather + traffic + status flags) and returned as JSON.
7. Client handles display-layer flags (whether weather/traffic/places should be displayed or not).

4. Components and responsibilities

Frontend (React) — View

Purpose	Collect the tour request and display the result
Responsibilities	Input form; loading state; map, stop list, weather chart, traffic panel; local layer toggles; language / planning-mode switches that trigger a new request
Depends on	Backend HTTP API
Why	No business logic in the UI → backend can evolve independently

Main components: `PlanningForm`,

`MapView`, `WeatherChart`,

`StopList`, `TrafficPanel`,

`LayerToggles`.

`TourController` — Controller

Purpose	HTTP boundary of the backend
Responsibilities	`GET /api/cities`, `GET /api/places`, `POST /api/tour`; validation; call `TourService`; return JSON / error responses
Depends on	`TourService`
Why	Keeps HTTP concerns out of planning code

`TourService` — Facade

Purpose	Single entry point for generating a tour and listing cities/places
Responsibilities	Call providers, handle per-source failure (`weatherStatus` / `trafficStatus`), choose strategy, build `TourResult`
Depends on	`WeatherProvider`, `TrafficProvider`, `PlaceRepository`, `TourStrategyFactory`
Why	Decouples the controller from subsystems; one place for partial-failure handling

`FmiWeatherAdapter` (implements `WeatherProvider`) — Adapter

Owner: Abdullah. Skeleton present; full implementation in progress.

Purpose	Convert FMI forecast into domain `HourlyForecast` objects
Responsibilities	Request one local day (UTC window for FMI); parse XML; convert times to Europe/Helsinki; extract temperature, precipitation, cloud cover, wind. Empty / NaN beyond horizon → empty list; HTTP/XML errors → `ProviderException`
Depends on	FMI WFS, `ApiHttpClient`, parser

`DigitrafficAdapter` (implements `TrafficProvider`) — Adapter

Skeleton present; full implementation in progress.

Purpose	Convert Digitraffic traffic announcements into domain objects
Responsibilities	Fetch announcements for the area; map to `TrafficAnnouncement` (type, description, geometry); empty list is a valid result
Depends on	Digitraffic API, `ApiHttpClient`

`JsonPlaceRepository` (implements `PlaceRepository`)

Purpose	Provide places and cities without an external places API
Responsibilities	Load `places.json` at startup; return cities and places by city id; support bilingual names (en/fi)
Why	No suitable open places API was found; change approved by the TA

`TourStrategy` / `TourStrategyFactory` — Strategy + Factory

Purpose	Select and order stops according to the chosen factors and weather
Responsibilities	Factory picks `GeneralTourStrategy` or `IndoorTourStrategy` from weather + factors; strategy produces a timed list of stops and a short explanation
Status	Interfaces, factory `choose()` and rain check implemented; `plan()` / `explanation()` still stubs

- **GeneralTourStrategy:** can use both indoor and outdoor places.
- **IndoorTourStrategy:** prefers indoor places first; adds outdoor only if time remains. Chosen when the weather factor is enabled and rain is expected (precipitation > 0.5 mm in any hour).

Shared HTTP infrastructure

`ApiHttpClient` + `ApiRequestFactory`: single client with timeouts, redirections and gzip support; creates URIs based on config; non-2xx status codes lead to `ApiClientException`. Adapters should concern themselves with the provider's data only.

Caching proxy (planned)

Short-term cache right in front of the providers to avoid repeating language and/or mode switching for the same city and date.

5. Data sources and design decisions

Candidate sources evaluated

Source	Role	Decision
FMI Open Data	Weather	**Kept** — free, no key, official
Digitraffic	Traffic	**Kept** — road announcements fit the UI better than train schedules
Open city / Visit data	Places	**Local JSON** — no single open API covered the needed places reliably; TA approved
OpenStreetMap	Map tiles / base map	Used on the frontend via Leaflet (no API key for core tiles)

Important design decisions

1. **Partial Failure is First-Class**

   In case weather or traffic fails, the tour will return back with the corresponding state (AVAILABLE / EMPTY / UNAVAILABLE). There is no need to show blank page, instead UI can display empty state or error state respectively.

2. **Factors vs Returned Data**

   Factors object determines what the planner may take into account. Weather and Traffic are requested in any case so that the user could switch display layers without making additional request.

3. **Places in the repository, not the external API**

   The choice was made because of lack of an appropriate public API. However, it is possible to provide an external API support in future through Repository API.

4. **Tour Planning in the Backend**

   All tour combination and planning remain in the backend. The UI is responsible for interactions and state managing around the TourResult contract.

5. **Strategy & Factory for tour creation**

   Two different strategies (general & indoor-first) reside in different classes. Using factory pattern, the if/else decision is hidden from TourService class. New strategies may be implemented easily without altering TourService facade.

6. **Demo profile**

   While adapters and strategies are unfinished, `spring.profiles.active=demo` serves fixed sample data so the full response shape can be tested and demoed.

6. API contract (summary)

Full contract:

`backend/docs/api-contract.md`.

Method	Path	Purpose
GET	`/api/cities`	Cities for the dropdown
GET	`/api/places?city=&language=`	Places of a city in one language
POST	`/api/tour`	Generate a tour

POST /api/tour body (required fields):

- `city` (id), `date` (YYYY-MM-DD), `startTime` (HH:mm), `durationHours` (1–12)
- optional: `language` (`en`|`fi`), `factors` (`WEATHER`, `TRAFFIC`)

Response includes: `stops`, hourly `weather`, `traffic` announcements, `weatherStatus`, `trafficStatus`, and the language/factors used.

CORS: `http://localhost:5173` is allowed by default.

7. Implementation status (midterm)

Backend

Area	Status
Project skeleton, Spring Boot, configuration	Done
Models, interfaces, HTTP client	Done
`TourController`, `TourService` (facade)	Done
`JsonPlaceRepository` + `places.json`	Done
`TourStrategyFactory.choose()` + rain check	Done
`TourStrategy.plan()` / `explanation()`	Stubs (logic draft exists)
`FmiWeatherAdapter`	Placeholder
`DigitrafficAdapter`	Placeholder
Demo profile with fixed sample output	Done
Caching proxy	Planned

Frontend

Area	Status
Planning form, layer toggles	Done
Map (Leaflet), stop list, weather chart, traffic panel	Done
“No traffic information” UI state	Done
Mock data in the expected backend shape	Done
Live calls to backend API	Not yet (next step)
Per-source loading indicators	Planned

The tables of implementation above are the proof of the progress; no completion percentage in terms of numbers is mentioned since the team has not developed the feature-weighted calculation method yet. The most crucial part that still needs to be done includes finishing up the external adapters, implementing the strategy algorithms, and tying up the frontend with the backend.

8. Self-evaluation

8.1 How well have we been able to stick to our original design?

Since we started discussing this project we have stayed with one approach, because the team has been communicating throughout. Our original design is still intact.

The one change we had to make was forced by the lack of an open API that provides tour information for all locations. We moved to storing place information in our own JSON file, and we asked our TA for approval of this change.

8.2 How well have we been able to implement features based on our original plan?

We have been able to implement features fairly well according to the original plan. The plan has not changed so far, so nothing has needed to be dropped or reworked.

What is implemented in the project snapshot reviewed for this document (verify against the final tagged commit):

- Backend skeleton: HTTP client, facade, controller, places repository, strategy factory and interfaces.
- Frontend screens and components that match the Figma prototypes and the API contract.
- Demo profile that returns a full `TourResult` shape for testing and demos.

What is still unfinished (as expected at midterm):

- Full FMI and Digitraffic adapters (still placeholders).
- Real stop-selection logic inside `plan()` / `explanation()` (stubs with a written draft).
- Frontend ↔ backend integration (UI still uses mock data).

8.3 How does our design correspond to quality?

Given what we have been presented with in the course, we believe we are using the right approaches and the correct patterns: Adapter, Facade and Proxy (with Strategy for stop selection).

We think this lets us reach better quality than building the application from simple classes. This matters especially because external APIs can fail, and we have already seen this happen. Our design handles a failing provider explicitly instead of letting the whole request fail.

The quality requirements are maintainability, testability, resilience and extensibility. Separate provider interfaces for parsing and orchestration allow the replacement or testing of an adapter. The `TourService` manages all combination logic and partial failures, and the `weatherStatus` and `trafficStatus` properties provide the UI with a means to differentiate between an unresponsive provider and an empty result. The repository interface abstracts away the curated JSON dataset, and different strategies make it easier to modify stop-selection rules.

These are **benefits of the design, not a demonstration of its quality**. The demo profile and smoke tests are some initial evidence that our controller and response format could function, but our real adapters, stop selection and frontend implementation remain unfinished and require integration testing. The caching Proxy hasn't been implemented yet, thus we have no way to measure any gains in performance.

8.4 What changes do we anticipate to implement the remaining features?

We expect to make a few changes here and there inside the Strategy part of the design, where the stop-selection logic is still being worked out.

We currently **do not anticipate** changing the public API contract or the main component boundaries, but integration may reveal changes that must then be recorded. The biggest delivery risk is that the UI, strategies and live provider adapters have so far been developed largely independently; the next milestone should be an end-to-end request using real responses, with failure and empty-data cases tested.

8.5 Individual tasks accomplished or changed per group member

Varun Ganesh Sondur

- I designed the repository structure. Chose the design patterns (Adapter, Strategy, Factory Method, Facade, layered architecture, Dependency Inversion). The caching Proxy is not built yet.
- I built the API Research Runner to test FMI, Digitraffic and VisitTampere. Its results led us to use our curated places data.
- I designed the components and interfaces (WeatherProvider, TrafficProvider, PlaceRepository, TourStrategy) and the shared models.

- I implemented the Spring Boot backend skeleton:
  - the HTTP client and request factory
  - the TourService facade, which keeps a failing provider from failing the request
  - the REST controller (/api/cities, /api/places, /api/tour) and structured error handling.
- I implemented JsonPlaceRepository and the seed places.json.
- I wrote the API contract, sample responses, PlantUML diagrams and the components and responsibilities document. The frontend and the other backend parts can build against them.
- I added a demo profile that swaps the stub adapters and strategy for fixed output so the backend can be smoke-tested before those parts are finished.

Thanh

- Created user interface prototypes in Figma (Option 1), consisting of: the planning screen, the loading screen, the results screen with data layer toggles, the results screen with one layer disabled, and the screen showing no traffic information.
- Optimized the screens after analyzing the APIs so that the UI matches the real data sources (road traffic alerts from Digitraffic instead of train schedules; places from the agreed data source rather than a live open places API).
- Implemented the React user interface using Vite, Tailwind CSS, Leaflet, and Recharts with the following screens/components: the planning form, data layer toggles, the map screen, the hourly weather graph, the stops list, and the road traffic information component.
- Added the ability to toggle layers so that Weather, Traffic, and Location data can be toggled on or off without having to make new API calls for data. The base map is still shown and layers control what is drawn on top of the base map.
- Added a "no traffic information" state where a notification is shown, the traffic panel is cleared out, and the related lines on the map are removed.
- Created the README file for the UI that includes the Node version requirement as well as instructions on how to install, run, and build the application.
- The UI at the moment uses sample data in the format that will be expected from the backend but is not connected to the Java backend. The next steps are: connecting the form to REST endpoints in the backend, replacing the sample data with API responses, and adding a loading bar for each data source.

Georgios Malezoglou

- Tested and proposed OpenStreetMap and made a small prototype to check how the map looks and if it's functional.
  - The map displays the desired country (zoomed in on Finland) on load, with different marks for every city where the user can click to zoom into a city.
  - I did that just to have an idea of how the map could look when a user loads it, and showed it to the team members.
  - The reason for choosing it is that it is entirely open-source and does not require an API key to access its core geographical data. Also, it looks simple and user friendly.
  - The prototype was made with the help of Qwen AI coder to speed up the prototype implementation process.
- Reviewed, discussed and agreed on Varun's proposed design patterns and the overall repository structure for the project.
- Responsible for the tour generation logic and the factory and strategy pattern implementation. Specifically:
  - Reviewed and agreed on the implementation of the `TourStrategy` (interface), `GeneralTourStrategy`, `IndoorTourStrategy` and `TourStrategyFactory` classes (where the whole tour generation logic,

factory and strategy must be implemented).
  - Strategy pattern: `GeneralTourStrategy` and `IndoorTourStrategy` have a common interface `TourStrategy` with `plan()` and `explanation()` methods, where:
    * `plan()` is meant to select the places that fit into the duration the user chose, using each place's visit time, and to return them as an ordered list of stops. Each stop is a place with a start and an end time. It is still a stub and the logic needs to be implemented.
    * `explanation()` returns a message explaining why a tour plan is chosen. It is still a stub and the logic needs to be implemented.
  - Factory pattern: in the `TourStrategyFactory` class, the `choose()` method is responsible for choosing between a tour strategy (general or indoor) based on 2 conditions (weather and expected rain). `IndoorTourStrategy` is returned only if the weather factor is enabled and rain is expected, otherwise `GeneralTourStrategy` is returned.
  - `IndoorTourStrategy` is meant to pick indoor places first and add outdoor places only if time remains, while `GeneralTourStrategy` can use both indoor and outdoor places.
  - Main flow: `TourService` asks the factory which strategy fits the forecast and the factors the user enabled. It then calls that strategy to select and sort the stops, and builds the final result from those stops, the explanation, the forecast and the traffic announcements. The forecast and announcements are always included in the result, whichever factors the user enabled.
  - Implemented the `TourStrategyFactory`'s `choose` method and created a separate (private) method `isRainExpected` that checks the hourly forecast and returns true if any hour has precipitation above the 0.5 mm threshold, otherwise false (I chose 0.5 mm for precipitation as the point where rain becomes noticeable. It is a constant and it might be changed later).
  - Reason for design pattern choice: Strategy is chosen so we can make separate classes for every rule (and add new possible rules) without touching `TourService`. Factory keeps the decision logic (if/else) out of `TourService`, in a separate class that chooses which strategy to use.
- Planned implementation idea for `GeneralTourStrategy`:
  - create `endOfTour` that calculates tour starting time + tour duration
  - add a 15 minute break (constant `VISIT_BREAK`) after a visit to a place is finished
  - look through the places from `places.json` in order with a for loop
  - for each place (stop), calculate the current time of the tour + the current place's visit duration + the visit break (`currentTime + visitMinutes + VISIT_BREAK`)
  - add an if statement checking whether there is room to add a place (stop) or not (checks that `currentTime + visitMinutes + VISIT_BREAK <= tour deadline`)
  - if the tour deadline is surpassed, the loop continues checking the next places from `places.json` in order and stops only after checking the last place (this might be changed later to stop when the tour's remaining time is less than the minimum visit duration (const `MIN_VISIT`

_DURATION), so the loop doesn't need to go on for no reason)
  - NOTE: when (and how) to add breaks between stops is yet to be decided and the constant value might be changed later. The idea described is still a draft.
- AI usage note: AI was used on the text of my evaluation to fix grammar and typo mistakes and to rephrase some sentences so they make more sense.

Abdullah

- Examined the FMI weather adapter and Digitraffic traffic adapter, including their provider interfaces and intended responsibilities.
- Reviewed how the adapters are designed to communicate with external APIs through the shared ApiHttpClient.
- Tested the backend API using GET /api/cities and POST /api/tour with the demo profile.
- Verified that the demo endpoint returns the expected tour response, including stops, weather information, traffic announcements, and status fields.
- Reviewed the current implementation and identified that the FMI and Digitraffic adapters are still placeholders and do not yet retrieve real external data.

9. How to run the application

Backend

Current limitation: The normal profile has placeholder external adapters and incomplete strategies. Use the `demo` profile to demonstrate the full response shape; this does not retrieve live FMI/Digitraffic data.

Requirements: Java 22, Maven 3.9+, free port 8080, internet (first dependency download + FMI/Digitraffic).

```
cd backend
mvn spring-boot:run
```

Or:

```
mvn clean package -DskipTests
java -jar target/city-tour-planner-backend-0.0.1-SNAPSHOT.jar
```

Smoke test:

```
curl http\://localhost:8080/api/cities
```

Demo profile (fixed sample data while adapters are unfinished):

```
mvn spring-boot:run -Dspring-boot.run.profiles=demo
```

Frontend

Requirements: Node.js 18+.

```
cd frontend/city-tour-ui
npm install
npm run dev
```

Opens at

`http://localhost:5173`.

Vite proxies `/api` to

`http://localhost:8080`.

Production build:

```
npm run build
npm run preview
```

More detail: root `README.md`,

`backend/README.md`,

`frontend/city-tour-ui/README.md`.

Appendix — Document locations in the repository

Content	Path
This design document	`midterm-design-document.md` (place in the submitted repository and update this path if moved)
API contract	`backend/docs/api-contract.md`
Component tables	`backend/docs/components-and-responsibilities.md`
PlantUML diagrams	`backend/docs/.puml`
Sample responses	`backend/docs/samples/`
Backend run instructions	`backend/README.md`
Frontend run instructions	`frontend/city-tour-ui/README.md`